☰ TOC

R3.05 - TD Thread et synchr

Objectif : Comprendre la concurrence, identifier les problèmes de race conditions et maîtriser les mécanismes de synchronisation en Java. Concept inspiré des cours de système : https://docs.oracle.com/javase/tutorial/essential/concurrency/

Cadre de travail

Vous utiliserez votre dépôt Git existant `OS-Nom-Prenom` pour organiser ce TD. L'objectif est de regrouper tous vos travaux pratiques de système dans une structure cohérente.

Organisation attendue :

```
OS-Nom-Prenom /              # Racine du dossier Git contenant tous les td et tp
├─ .gitignore                # fichiers/éléments à ignorer par Git
├─ README.md                 # informations additionnelles
├─ LectureEcritureJava/      # TP précédent sur les I/O
├─ .../                      # autres TDs/TPs
├─ Parallelisation/          # ← CE TD
│  ├─ README.md              # spécifique à cet exercice
│  ├─ CalculSequentiel.java
│  ├─ Calcule.java
│  ├─ CalculParallele.java
│  ├─ CompteurSecurise.java
│  ├─ CacheSecurise.java
│  ├─ CompteurDangereux.java
│  ├─ ProducteurConsommateur.java
│  ├─ TestCompteur.java
│  ├─ TestMutex.java
│  ├─ TestRWLock.java
│  └─ TestSemaphore.java
```

Parallélisation d'un calcul

Dans cet exercice, vous allez transformer un algorithme séquentiel en version parallèle pour comprendre les bénéfices de la concurrence sur les **opérations en lecture seule**. L'idée fondamentale est que lorsque plusieurs threads ne font que **lire** des données partagées sans les modifier, il n'y a aucun problème de concurrence. C'est le scénario idéal du multithreading : diviser le travail sans conflits : maximum de performances !

Créez `CalculSequentiel.java` :

Le code suivant simule un traitement intensif sur un tableau de données. Chaque élément nécessite un calcul "coûteux" en CPU. Dans la version séquentielle, ces calculs se font un par un.

```java
public class CalculSequentiel {
    private static final int[] DONNEES = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    private static final int MULTIPLICATEUR = 1000;

    public static void main(String[] args) {
        System.out.println("=== Calcul Séquentiel ===");

        long debut = System.nanoTime();
        long sommeTotal = 0;

        // TODO: Ce code sera remplacé par une version parallèle
        for (int i = 0; i < DONNEES.length; i++) {
            int valeur = DONNEES[i];

            // Calcul intensif simulé par des opérations mathématiques
            long resultat = 0;
            for (int j = 0; j < MULTIPLICATEUR; j++) {
                resultat += valeur * valeur + valeur;
                            // on pourrais aussi mettre un sleep
            }

            sommeTotal += resultat;
            System.out.println("Traitement de " + valeur + " terminé : " + resultat);
        }

        long duree = (System.nanoTime() - debut) / 1_000_000;
        System.out.println("Résultat total : " + sommeTotal);
        System.out.println("Durée : " + duree + " ms");
```

```
    }
}
```

Observations importantes :

- Le tableau `DONNEES` est déclaré `static final` : il ne change jamais
- Chaque calcul est **indépendant** : le traitement de l'élément i n'affecte pas l'élément j
- La variable `sommeTotal` accumule les résultats : c'est ici qu'il faudra être prudent en version parallèle

Créer la classe Thread personnalisée

Créez `Calcule.java` en suivant ces étapes : Appuyez-vous sur la documentation Creating and Starting Threads.

La classe `Thread` en Java représente un fil d'exécution. En héritant de `Thread`, vous créez votre propre type de thread spécialisé dans un calcul particulier.

Étape 1 : Créez une classe `Calcule` qui **hérite de Thread**

```
public class Calcule extends Thread {
    // TODO: Déclarer les attributs nécessaires
}
```

Pourquoi hériter de Thread ? Cela vous donne accès à toutes les méthodes de gestion des threads (start(), join(), etc.) et vous oblige à implémenter la méthode run() qui contient le code à exécuter.

Étape 2 : Ajoutez un **constructeur** qui prend en paramètre :

- La valeur à traiter (`int valeur`) : chaque thread travaillera sur une donnée spécifique
- Le multiplicateur (`int multiplicateur`) : pour contrôler l'intensité du calcul
- Un index pour identifier le thread (`int index`) : utile pour le debugging et l'affichage

Le constructeur sert à **configurer** chaque thread avec ses données de travail. Chaque thread aura sa propre **copie** de ces valeurs, stockées dans ses attributs privés.

Étape 3 : Implémentez la méthode `run()` avec l'annotation `@Override`

```
@Override
public void run() {
    // TODO: Implémenter le calcul intensif
    // Même logique que dans la boucle interne de CalculSequentiel
}
```

Rôle de run() : C'est le "programme principal" de votre thread. Quand vous appelez `start()` sur un thread, Java créé un nouveau fil d'exécution et y lance la méthode `run()`. Tout le travail du thread se fait ici.

Étape 4 : Ajoutez une méthode `getResultat()` pour récupérer le résultat du calcul.

Pourquoi cette méthode ? En programmation concurrente, il faut un moyen pour le thread principal de récupérer les résultats calculés par chaque thread worker. Cette méthode permet de "récolter" les résultats après que tous les threads aient terminé.

Version parallèle avec Thread

Créez `CalculParallele.java`

Maintenant vous allez orchestrer plusieurs threads pour diviser le travail. Le principe : au lieu de traiter les 10 éléments en séquence, créer 10 threads qui traitent chacun un élément en parallèle.

Étape 1 : Initialisez un **tableau de threads**

```
Calcule[] threads = new Calcule[DONNEES.length];
```

Pourquoi un tableau ? Vous devez garder une référence vers chaque thread pour pouvoir les contrôler (les démarrer, attendre leur fin, récupérer leurs résultats). Le tableau structure cette gestion.

Étape 2 : Remplacez la boucle séquentielle par :

- Création des instances de `Calcule` dans une boucle
- Stockage dans le tableau de threads `Calcule threads[] = new Calcule[...]`

Logique : Au lieu d'exécuter le calcul directement, vous créez un "ouvrier" (thread) spécialisé pour chaque tâche. Chaque thread reçoit ses paramètres de travail via le constructeur.

Étape 3 : Démarrez tous les threads avec une boucle utilisant `.start()`

> Documentation : Thread.start() vs run() - Pourquoi utiliser `start()` et pas `run()` directement ? `start()` demande au système d'exploitation de créer un nouveau fil d'exécution et d'y lancer `run()`. Si vous appelez `run()` directement, le code s'exécute dans le thread courant (pas de parallélisme) !

Séquence critique : Après `start()`, chaque thread commence à s'exécuter **immédiatement** et **en parallèle**. Le thread principal continue son exécution pendant que les threads workers calculent.

Étape 4 : Attendez la fin de tous les threads avec `.join()` dans une boucle

Pourquoi join() ? Le thread principal doit attendre que tous les workers terminent avant de pouvoir collecter les résultats. `join()` bloque le thread courant jusqu'à ce que le thread cible termine.

Étape 5 : Récupérez et additionnez tous les résultats

Point critique : C'est ici qu'il faut être prudent ! Vous additionnez les résultats dans le thread principal, **après** que tous les threads workers aient terminé. Il n'y a donc pas de concurrence sur l'addition.

Questions d'analyse

Après implémentation, répondez à ces questions :

1. **Performance :** Quelle version est la plus rapide ? Pourquoi ?
   - *Attendu :* Version parallèle plus rapide grâce à l'utilisation de plusieurs cœurs CPU
   - *Oui mais :* Actuelement les calcules prenent de base peu de temps, et la gestion des threads rajoute du temps
   - Au final la nouvelle version peu prendre un peu plus de temps que prévue, mais dans une situation ou les calcules prennent beaucoup de temps -> c'est largement mieux.
   - Dans ce cas d'utilisation les ExecutorService et Callable sont à préviligier.
2. **Résultats :** Les résultats sont-ils identiques entre les versions ? Pourquoi n'y a-t-il pas de race condition ?
   - *Point clé :* Chaque thread travaille sur des données différentes et l'agrégation se fait de manière séquentielle

Race conditions

Démontrer que les **écritures concurrentes** sur des variables partagées créent des comportements imprévisibles appelés **race conditions**. Documentation : Memory Consistency Errors - Oracle Java Tutorial.

Contrairement à l'exercice précédent où chaque thread avait ses propres données, ici **tous les threads modifient la même variable**. C'est le scénario classique qui génère des bugs de concurrence.

Créez `CompteurDangereux.java` : C'est le code de base problématique

```java
public class CompteurDangereux {
    private static int compteurGlobal = 0;  // Variable partagée MODIFIABLE

    static class Incrementeur implements Runnable {
        private final String nom;
        private final int nombreIncrements;

        public Incrementeur(String nom, int nombreIncrements) {
            this.nom = nom;
            this.nombreIncrements = nombreIncrements;
        }

        @Override
        public void run() {
            for (int i = 0; i < nombreIncrements; i++) {
                // TODO: Expliquer pourquoi cette ligne pose problème
                compteurGlobal++;  // RACE CONDITION ICI !
            }
            System.out.println(nom + " terminé. Compteur vu : " + compteurGlobal);
        }
    }

    public static int getCompteur() {
        return compteurGlobal;
    }

    public static void resetCompteur() {
        compteurGlobal = 0;
    }
}
```

Analyse du problème

La variable `compteurGlobal` est partagée entre tous les threads. Contrairement au tableau `DONNEES` de l'exercice 1 qui était en **lecture seule**, ici chaque thread **modifie** cette variable.

TODO : Créez `TestCompteur.java` :

Ce test va révéler le non-déterminisme des race conditions. Plus vous créez de contention (threads nombreux + opérations nombreuses), plus les erreurs apparaissent. Il y a plus de chance que plusieurs threads modifie "simultanément" la variable.

Étape 1 : Créez une boucle qui répète le test **50 fois** pour observer la variabilité

Pourquoi 50 fois ? Les race conditions sont **probabilistes**. Parfois le programme donne le bon résultat par chance. En répétant, vous verrez la variabilité des résultats. Si ce n'est pas assé, augmenté.

Étape 2 : Pour chaque test :

- Réinitialisez le compteur avec `resetCompteur()` : *état propre à chaque test*
- Créez **9 threads** qui incrémentent chacun **1111111 fois** : *gros volume pour maximiser les conflits*
- Démarrez tous les threads et attendez leur fin avec `join()` : *orchestration classique*

Calcul attendu : 9 threads × 1111111 incréments = 9999999 total

Étape 3 : Affichez le résultat final et comparez avec la valeur attendue (1111111 * 9)

Observation typique : Vous devriez voir des résultats variables, souvent inférieurs à 9999999. Plus il y a de threads, plus les "pertes" sont fréquentes.

Étape 4 : Ajoutez des statistiques :

- Comptez combien de fois le résultat est correct : *mesure de fiabilité*
- Calculez le pourcentage de réussite : *visualisation de l'ampleur du problème*
- Affichez la valeur minimale et maximale obtenue : *étendue de la variabilité*
- Changez le nombre de threads (1, 2, 5, 10) - Impact sur la fréquence des race conditions ? : *relation contention/erreur*
- Changez le nombre d'incréments par thread - Plus d'opérations = plus de problèmes ? : *volume/erreur*

Question clé : Pourquoi l'opération `compteurGlobal++` n'est-elle pas atomique ?
Cette opération se décompose en trois étapes CPU :
1. LOAD : Lecture de la valeur actuelle de `compteurGlobal` dans un registre
2. ADD : Incrémentation de la valeur dans le registre
3. STORE : Écriture de la nouvelle valeur dans `compteurGlobal`
Le problème : Si deux threads exécutent ces étapes en même temps, ils peuvent tous les deux lire la même valeur initiale, l'incrémenter séparément, et l'un "écrase" le travail de l'autre lors du STORE ! Voir diagram sur le cours.

Séquence problématique typique :

1. Thread A : lit compteur = 100
2. Thread B : lit compteur = 100 (même valeur !)
3. Thread A : calcule 100 + 1 = 101
4. Thread B : calcule 100 + 1 = 101
5. Thread A : écrit 101
6. Thread B : écrit 101 (écrase le travail de A !)
7. Résultat : compteur = 101 au lieu de 102

Protection avec Mutex (synchronized)

Un **mutex** (mutual exclusion) garantit qu'un seul thread peut exécuter une section critique à la fois. En Java, chaque objet possède un **verrou intrinsèque** (intrinsic lock). Le mot-clé `synchronized` utilise ce verrou, c'est notre mutex. Voir Synchronized Methods et Intrinsic Locks

Avant d'entrer dans cette section critique, un thread doit acquérir le verrou associé au mutex. Si ce verrou est déjà détenu par un autre thread, le thread courant est placé en attente jusqu'à ce que le verrou soit libéré. Lorsque le thread propriétaire quitte la section critique, il libère le verrou, autorisant ainsi un autre thread en attente à poursuivre son exécution et à acquérir le mutex à son tour.



Dans les implémentations classiques, ce mécanisme repose sur des fonctions système telles que `mutex_lock` et `mutex_unlock`. Toutefois, le langage Java propose une abstraction de ce processus à travers le mot-clé synchronized. L'acquisition du verrou est alors effectuée implicitement au début du bloc synchronized, et sa libération intervient automatiquement à la fin de ce bloc, simplifiant ainsi la gestion explicite de la synchronisation par le programmeur ...

TODO : Créez `CompteurSecurise.java` :

Étape 1 : Reprenez le code de `CompteurDangereux` comme base

Transformation : Vous allez protéger les accès concurrents à `compteurGlobal` sans changer la logique métier.

Étape 2 : Ajoutez un **objet verrou** explicite

```
private static final Object verrou = new Object();
```

Pourquoi un objet séparé ? Utiliser un objet dédié comme verrou est une bonne pratique pour éviter les interactions avec d'autres synchronisations : On peut vouloir différencier différents verrous au sein d'une même classe / comportement.
Alternative : synchroniser directement sur la classe avec `synchronized(CompteurSecurise.class)` mais c'est moins flexible.

Étape 3 : Protégez la section critique dans la méthode `run()`

```
synchronized(verrou) {
    // TODO: Placer ici l'opération critique
    compteurGlobal++;
}
```

Mécanisme :

- **Avant** d'entrer dans le bloc, le thread tente d'acquérir le verrou
- Si le verrou est libre → acquisition immédiate + entrée dans le bloc
- Si le verrou est occupé → le thread **bloque** et attend
- **À la sortie** du bloc (normale ou par exception), le verrou est **automatiquement libéré**

Étape 4 : Protégez également les méthodes `getCompteur()` et `resetCompteur()` **Pourquoi ?** Même les **lectures** doivent être protégées ! Si un thread lit `compteurGlobal` pendant qu'un autre l'écrit, la lecture peut récupérer une valeur incohérente.

```
public static int getCompteur() {
    synchronized(verrou) {
        return compteurGlobal;
    }
}

public static void resetCompteur() {
    synchronized(verrou) {
        compteurGlobal = 0;
```

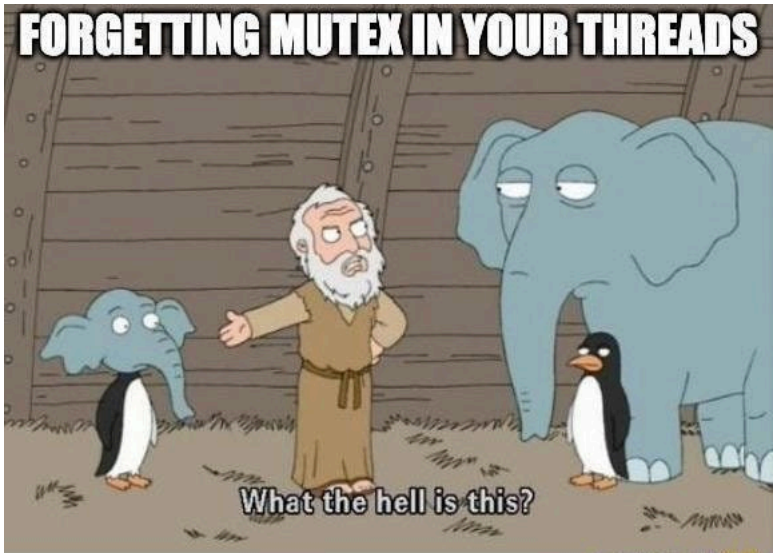
```
    }
}
```

Étape 5 : Reprenez le test de l'exercice 2 mais avec la version sécurisée pour créer `TestMutex.java`

Observation attendue : Vous devriez maintenant **toujours** obtenir le résultat attendu (9999999). Plus de variabilité, plus de "pertes".

Coût : La version synchronisée sera **plus lente** car les threads doivent attendre leur tour au lieu de travailler en parallèle. C'est le compromis sécurité/performance.

Obtenez-vous maintenant toujours le même résultat attendu ?



Sémaphore (producteur-consommateur)

Les sémaphores généralisent les mutex : au lieu de permettre 1 seul accès simultané, ils permettent N accès simultanés. C'est parfait pour gérer des ressources limitées (connexions DB, threads workers, buffer borné) mais un poil plus lent.

> Attention, il est possible de recrée le comportement d'un mutex avec un sémaphore dit binnaire. C'est une mauvaise pratique.

Contexte : Un buffer de taille limitée partagé entre :

- **Producteurs** : Ajoutent des éléments (bloqués si buffer plein)
- **Consommateurs** : Retirent des éléments (bloqués si buffer vide)

Défis de synchronisation :

- Éviter que les producteurs remplissent un buffer plein : *contrôle de flux*
- Éviter que les consommateurs vident un buffer vide : *synchronisation production/consommation*
- Protéger l'accès concurrent au buffer : *exclusion mutuelle sur la structure de données*

Architecture de la solution : Nous utiliserons **3 sémaphores** pour résoudre ces 3 problèmes indépendamment.

TODO : Créez `ProducteurConsommateur.java` :

Étape 1 : Déclarez les sémaphores et la structure de données

```
import java.util.concurrent.Semaphore;
import java.util.LinkedList;
import java.util.Queue;

private static final int TAILLE_BUFFER = 5;
private final Integer[] buffer = new Integer[TAILLE_BUFFER];

// TODO: Déclarer 3 sémaphores avec les bonnes valeurs initiales
private final Semaphore placesLibres = new Semaphore(???);
private final Semaphore elementsDisponibles = new Semaphore(???);
private final Semaphore mutexBuffer = new Semaphore(???); // quel erreur ici ? vous pouvez le résoudre plus tard
```

Question : Quelles valeurs initiales pour chaque sémaphore ?
- `placesLibres` : TAILLE_BUFFER (5) - Au début, le buffer est vide donc 5 places libres
- `elementsDisponibles` : 0 - Au début, aucun élément à consommer
- `mutexBuffer` : 1 - Un seul thread peut manipuler le buffer à la fois (mutex classique)

Logique : Les deux premiers sémaphores s'équilibrent : placesLibres + elementsDisponibles = TAILLE_BUFFER toujours !

Étape 2 : Implémentez la classe Producteur

```
class Producteur implements Runnable {
    public void run() {
        for (int i = 0; i < 10; i++) {
            int produit = genererProduit();

            // TODO: Attendre une place libre
            // TODO: Acquérir l'accès exclusif au buffer
```

```
            // TODO: Ajouter l'élément au buffer
            // TODO: Libérer l'accès au buffer
            // TODO: Signaler qu'un élément est disponible
        }
    }
}
```

Séquence logique du producteur :

1. Réservez une place (bloque si buffer plein)
2. Bloquez la section
3. Ajout réel de l'élément
4. Débloqué la section
5. Signaler aux consommateurs

Pourquoi cet ordre ? Acquérir d'abord `Réservez` évite de bloquer la section inutilement si le buffer est plein. C'est donc plus efficace. A contrario on pourrais resté bloqué.

Étape 3 : Implémentez la classe Consommateur

```
class Consommateur implements Runnable {
    public void run() {
        for (int i = 0; i < 7; i++) {
            // TODO: Attendre qu'un élément soit disponible
            // TODO: Acquérir l'accès exclusif au buffer
            // TODO: Retirer un élément du buffer
            // TODO: Libérer l'accès au buffer
            // TODO: Signaler qu'une place est libre
        }
    }
}
```

Séquence logique du consommateur :

1. Attendez un élément (bloque si buffer vide)
2. Accès exclusif au buffer
3. `Integer produit = buffer.poll()` : Retrait réel de l'élément
4. Libération accès buffer
5. Signaler aux producteurs

Symétrie : Le consommateur fait exactement l'inverse du producteur. Chaque `acquisition` du semaphore du producteur correspond à un `release` du semaphore consommateur et vice-versa.

Test et validation

Créez `TestSemaphore.java` avec ces scénarios :

- **Scénario équilibré :**
  - 2 producteurs, 3 consommateurs
  - Vérifiez que tous les produits sont consommés
  - *Observation :* Le système doit fonctionner sans blocage
- **Scénario déséquilibré :**
  - 5 producteurs, 2 consommateurs
  - Observez le comportement avec déséquilibre
  - *Observation :* Le buffer va se remplir rapidement, les producteurs vont attendre
- **Monitoring :**
  - Ajoutez des compteurs pour suivre la production/consommation
  - Affichez l'état du buffer périodiquement
  - *But :* Visualiser la dynamique du système et vérifier la cohérence

Questions d'analyse importantes

1. **Ordre des acquire()** : Que se passe-t-il si on inverse `placesLibres.acquire()` et `mutexBuffer.acquire()` ?
2. **Cohérence** : Les compteurs de production/consommation s'équilibrent-ils ?
3. **Blocages** : Dans quelles conditions le système peut-il se bloquer ?

Cette architecture avec sémaphores est un **pattern classique** de la programmation concurrente, réutilisable dans de nombreux contextes (pools de ressources, pipelines de traitement, impression, etc.).

© Jehan-Antoine Vayssade 2011-2025